

Compiler Construction (CS4031)

Date: September 22<sup>nd</sup> 2026

Course Instructor(s)

Farooq Ahmad, Faisal Aslam,

Aamir Raheem, Hamad Ulqudous

Sessional-I Exam

Total Time (Hrs): 1

Total Marks: 40

Total Questions: 4

Roll No

Section

Student Signature

Do not write below this line

Attempt all the questions.

*CLO 1: Describe the basic phases of compilation and recognize the theories and techniques used in each phase.*

Q 1: Regular Expressions (10 marks).

A configuration language has the following three lexeme categories (whitespace — spaces and tabs — separates lexemes and is not part of any of them):

- **Identifiers:** an identifier *must begin with a letter*. After that it may contain letters, digits and underscores, subject to one rule: **every underscore must be immediately preceded and immediately followed by a letter or a digit**. Thus `buf_size_2` and `x1` are valid, whereas `_x` (leading underscore), `x_` (trailing underscore), `a__b` (two consecutive underscores) and `_` on its own are *not* valid identifiers.
- **Duration literals:** one or more digits followed by a **mandatory** unit suffix — `ms` (milliseconds), `s` (seconds), or `m` (minutes). A digit string with no unit after it (e.g. `250`) is *not* a valid lexeme. **Note:** since identifiers must start with a letter, `ms` on its own (with no digits before it) is an *identifier*, not a duration literal — a unit suffix only counts as one when it immediately follows a digit string.
- **Operators:** `:` (type annotation), `::` (scope resolution), and `:=` (assignment).

Let the alphabet be

$$\Sigma = \{a, b, c, \dots, z, 0, 1, 2, \dots, 9, \_, :, =\},$$

and let

$$L = [a-z], \quad D = [0-9].$$

Write a regular expression for *each* of the three lexeme categories, using **only** the symbols of  $\Sigma$  (you may use  $L$  and  $D$  as shorthand). Each regular expression must generate *exactly* the valid lexemes of its category and nothing else.

- $R_{id}$  : identifiers (mind the underscore rule — your expression must not generate `_x`, `x_`, `a__b` or `_`).

National University of Computer and Emerging Sciences  
Lahore Campus

- $R_{dur}$  : duration literals (the unit is mandatory).
- $R_{op}$  : operators.

*CLO 1: Describe the basic phases of compilation and recognize the theories and techniques used in each phase.*

---

**Q 2: Transition Diagram (10 marks).**

A logging library tags each record with three kinds of lexeme. Below is the regular expression for each, over the alphabet of printable ASCII characters, where  $D = [0-9]$ ,  $L = [a-zA-Z]$ , and  $S$  denotes a single space character:

- $R_{LEVEL} = \text{INFO} \mid \text{WARN} \mid \text{ERROR}$  — token LEVEL; attribute is which of the three keywords matched.
- $R_{TIME} = \text{DD}:\text{DD}:\text{DD}$ , where each D stands for a single digit — i.e. exactly two digits, a colon, two digits, a colon, two digits (an HH:MM:SS timestamp, e.g. 14:05:09). Token TIME; attribute is the matched text.
- $R_{STR} = "(L \mid D \mid S)^*" — a double-quoted message containing any mix of letters, digits, and spaces, e.g. "disk 2 almost full". Token STR; attribute is the matched text (without the surrounding quotes).$

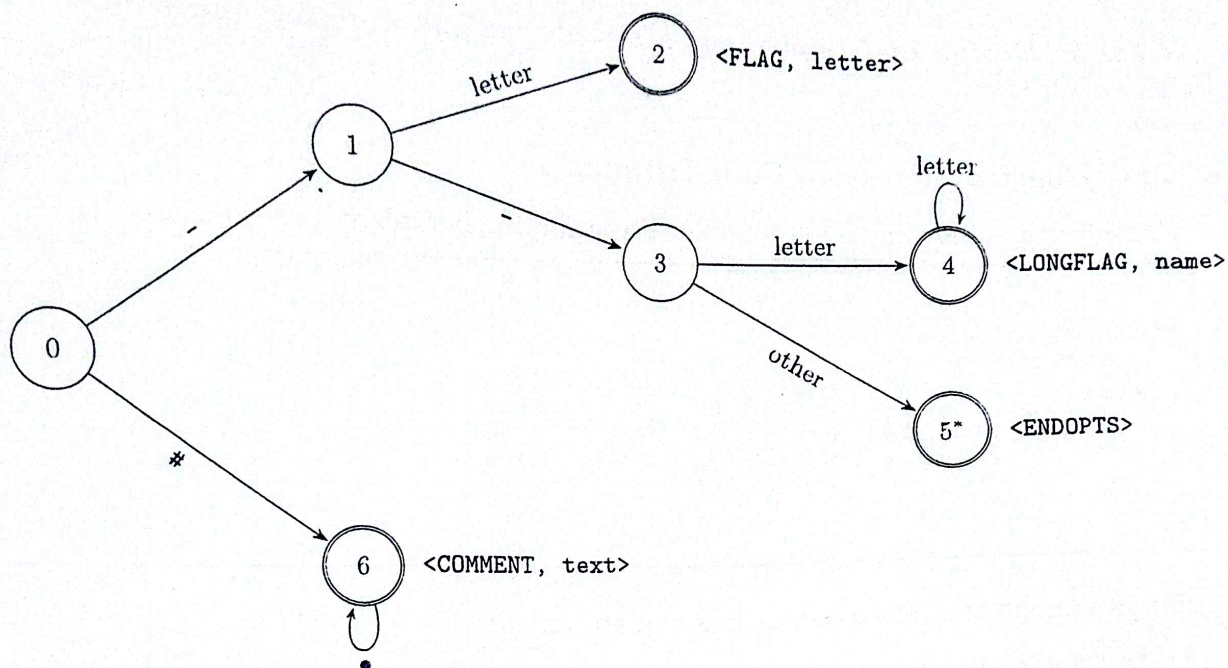
Draw **one** transition diagram (DFA) that recognizes all three of the above lexeme categories. Follow the notation given at the top of this exam.

*CLO 3: Design and implement a simple lexical analyzer and parser for a given CFG.*

---

**Q 3: From Diagram to Code (10 marks).**

A command-line parser recognizes four kinds of lexeme, exactly as drawn in the transition diagram below (built to the notation given at the top of this exam). As is standard for regular expressions, the symbol  $\cdot$  (a dot) is used below to mean "any single character other than a newline".



- FLAG: a single lowercase letter immediately after one -, e.g. -v, -o.
- LONGFLAG: one or more lowercase letters immediately after -, e.g. -verbose.
- ENDOPTS: exactly two dashes - with no letter immediately after (the conventional "no more flags follow" marker). State 5 is starred: the character after the second dash is read to check whether it is a letter, and if it is not, that character was not actually part of this token and must be retracted before ENDOPTS is returned.
- COMMENT: a # followed by zero or more . (any character except newline). up to (but not including) the next newline.

Complete the function below so it scans exactly one token starting at input[\*pos], skips leading whitespace, advances \*pos past the lexeme it accepts, and returns the token. **You must design your own representation for a token** (how you name and store each token's name and attribute, e.g. Token below could be a struct you define) — none is given. Follow the states of the diagram above; do not use regex.h or similar library functions.

```

1  /* Define any type(s) you need to represent a token above this
   point
2  (e.g. a struct holding a token name and an attribute). */
3
4  Token nextToken(const char *input, int *pos) {
5      /* input : pointer to the null-terminated source buffer being
6       * scanned.
7       * pos   : in/out -- current scanning position in input. Skip
8       * leading whitespace, then advance *pos past the
9       * lexeme
10      * you accept, so the next call resumes right after
11      * it.
12      */
13      /* your code here */
14  }

```

National University of Computer and Emerging Sciences  
Lahore Campus

*CLO 3: Design and implement a simple lexical analyzer and parser for a given CFG.*

**Q 4: CFG Construction from Code (10 marks).**

Consider the following C++ fragment (assume *x* has already been declared).

```
1 x = 5;  
2 if (x > 0) {  
3     x = x - 1;  
4 } else {  
5     while (x < 10) {  
6         x = x + 1;  
7     }  
8 }
```

This fragment uses exactly four kinds of statement:

- (a) an **assignment**, e.g. *x* = 5;
- (b) an **if-else** statement,
- (c) a **while** statement,
- (d) a **block**: zero or more statements enclosed in { }.

Write a context-free grammar with start symbol *Stmt* that generates these four statement forms, under the following conventions:

- **Terminals available to you:** *id*, *=*, *;*, *(*, *)*, *{*, *}*, *if*, *else*, *while*.
- **Assume two nonterminals are already defined for you:** *Expr* (an arithmetic expression such as 5 or *x* - 1) and *Cond* (a comparison such as *x* > 0). *Do not write productions for Expr or Cond* — just use them as symbols on the right-hand side. No marks are awarded for defining them.
- The body of an if-branch, an else-branch, and a while-loop is always a block in { } (never a single bare statement).
- Every if statement has an else branch.
- You do **not** need to eliminate left recursion, left-factor, or make the grammar LL(1). Ambiguity is acceptable. Any grammar generating exactly these four forms earns full marks.